

1. Creating and Manipulating R Objects

Vectors

Theory: A vector in R is a one-dimensional array that contains elements of the same type (numeric, character, or logical). Operations on vectors are often vectorized for efficiency.

- **c()**: Combines values into a vector. It is the basic function for creating vectors in R.

```
v <- c(1, 2, 3, 4, 5)
```

- **print()**: Displays the value of an object.

```
print(v)
```

Matrices

Theory: A matrix is a two-dimensional array where every element has the same type. Matrices are useful for performing mathematical operations on data arranged in rows and columns.

- **matrix()**: Creates a matrix from a given set of values. The nrow and byrow parameters specify the number of rows and whether to fill the matrix by rows or columns.

```
m <- matrix(1:9, nrow = 3, ncol = 3)
```

- **print()**: Displays the matrix or any other object.

```
print(m)
```

```
m[2,3]
```

Arrays

Theory: Arrays are multi-dimensional extensions of matrices that can hold data in multiple dimensions. The number of dimensions is determined when the array is created.

- **array()**: Creates an array with multiple dimensions. The dim parameter specifies the dimensions of the array.

```
a <- array(1:12, dim = c(3, 2, 2))
```

- **print()**: Displays the array.

```
print(a)
```

Data Frames

Theory: A data frame is a two-dimensional table where each column can contain different types of data (e.g., numeric, character). Data frames are commonly used in R for storing datasets.

- **data.frame()**: Creates a data frame from vectors of equal length. Columns can be of different types (numeric, character, etc.).

```
df <- data.frame(Name = c("Alice", "Bob", "Charlie"), Age = c(25, 30, 35))
```

- **print()**: Displays the data frame.

```
print(df)
```

Lists

Theory: Lists are collections of different types of objects (vectors, matrices, data frames, etc.). Lists can store heterogeneous elements, making them very flexible.

- **list()**: Creates a list, which can hold elements of different types and structures.

```
l <- list(Name = "Alice", Age = 25, Scores = c(90, 85, 88))
```

- **print()**: Displays the list.

```
print(l)
```

2. Demonstrating Loops & Vectorization, Missing Values

Loops

Theory: Loops allow you to repeat a block of code multiple times. In R, for, while, and repeat loops are used for iteration.

- **for loop**: A control flow statement for iterating over a sequence of values.

```
l <- 1
```

```
While(i<=10){
```

```
  If(i==5){
```

```
    l<-i+1
```

```
  Next
```

```
}
```

```
Print(i)
```

```
l<-i+1
```

```
}
```

Vectorization

Theory: R is designed to handle operations on entire vectors without the need for explicit loops. This is called vectorization and makes computations faster.

- **Vectorized operations**: R performs operations on vectors element-wise, making computations more efficient.

```
v <- c(1, 2, 3, 4, 5)
```

```
v <- v * 2
```

Missing Values

Theory: In datasets, some values may be missing (represented by NA). R provides functions like is.na() to check for missing values and na.omit() to remove them.

- **is.na()**: Checks for NA (missing) values in a vector or data frame.

```
print(is.na(v))
```

- **na.omit()**: Removes NA values from a vector or data frame.

```
v <- na.omit(v)
```

```
v<-c(1,2,3,NA,5)
```

3. Importing and Exporting Data

Importing and exporting data involves reading data from external files (e.g., CSV, Excel) into R for analysis and saving results back to files.

Importing Data

- **read.csv()**: Reads data from a CSV file into a data frame.

```
df <- read.csv("path/data.csv")
```

```
head(df)
```

```
install.packages(readxl)
```

```
library(readxl)
```

```
data <- read_excel("path/data.xlsx")
```

Exporting Data

- **write.csv()**: Writes data from a data frame to a CSV file. `row.names = FALSE` prevents writing row names.

```
write.csv(df, "output.csv", row.names = FALSE)
```

4. Data Manipulations

Summarizing

Theory: Summarizing data gives you a quick statistical overview of your dataset, such as mean, median, and variance.

- **summary()**: Provides a summary of statistics for each column in a data frame (mean, median, quartiles, etc.).

```
summary(df)
```

Sorting

Theory: Sorting is arranging data in a particular order (ascending or descending) based on one or more variables.

- **order()**: Returns the order of elements in a vector. Used to sort data frames.

```
df_sorted <- df[order(df$Age), ]
```

Subsetting

Theory: Subsetting involves extracting parts of the data frame based on specific conditions.

- **subset()**: Extracts subsets of a data frame based on conditions.

```
df_subset <- subset(df, Age > 30)
```

Merging and Joining

Theory: Merging or joining combines two data frames based on a common key or variable.

- **merge()**: Combines two data frames by common columns or row names.

```
df_merged <- merge(df, df2, by = "Name")
```

- **left_join()** (from dplyr): Joins two data frames by common columns, keeping all rows from the left data frame.

```
library(dplyr)
```

```
df_joined <- left_join(df, df2, by = "Name")
```

Statistical Hypothesis Generation and Testing: Why, When, and How: Statistical hypothesis generation and testing is used to determine whether there is enough evidence to support or reject a hypothesis about a dataset. It's an essential part of inferential statistics, allowing us to make conclusions about a population based on a sample.

1. Chi-Square Test

- **What:** A test for independence between two categorical variables.
- **Why:** To determine if there's a significant association between two categorical variables.
- **When:** Use when you have two categorical variables and want to test whether they are related (e.g., gender and department).
- **How:** It compares observed frequencies with expected frequencies assuming no association, resulting in a chi-square statistic.

```
chisq.test(table(df$Gender, df$Department))
```

- **Expected Range:**
 - Chi-Square statistic: ≥ 0 (non-negative).
 - p-value: 0 to 1. A value < 0.05 indicates a significant relationship.
-

2. t-Test

- **What:** A test for comparing the means of two groups.
- **Why:** To assess if the means of two groups are statistically different.
- **When:** Use when you want to compare the mean of a numeric variable between two groups (e.g., exam scores of male and female students).
- **How:** The test calculates the difference in means and compares it against variability in the data, yielding a t-statistic and p-value.

```
t.test(Score ~ Gender, data = df)
```

- **Expected Range:**
 - t-statistic: Typically ranges from -4 to 4 (can be larger).
 - p-value: 0 to 1. A value < 0.05 suggests a significant difference between means.
-

3. Analysis of Variance (ANOVA)

- **What:** A test to compare the means of more than two groups.
- **Why:** To determine whether the means of multiple groups are significantly different.
- **When:** Use when comparing means across more than two groups (e.g., comparing exam scores across multiple departments).
- **How:** ANOVA calculates the variance between group means and within groups, yielding an F-statistic.

```
aov(Score ~ Department, data = df)
```

- **Expected Range:**
 - F-statistic: ≥ 0 (non-negative).
 - p-value: 0 to 1. A value < 0.05 indicates at least one group mean is significantly different.

4. Correlation Analysis

- **What:** A test that measures the strength and direction of the linear relationship between two continuous variables.
- **Why:** To quantify how strongly two variables are related.
- **When:** Use when you want to check the linear relationship between two continuous variables (e.g., age and score).
- **How:** The correlation coefficient (r) measures the direction and strength of the relationship, along with a p-value for significance.

`cor.test(df$Age, df$Score)`

- **Expected Range:**
 - Correlation coefficient (r): -1 to 1.
 - p-value: 0 to 1. A value < 0.05 suggests a significant correlation.
-

5. Maximum Likelihood Estimation (MLE)

- **What:** A method for estimating parameters that maximize the likelihood of observing the data.
- **Why:** To estimate model parameters (like mean and variance) for the best fit to the data.
- **When:** Use when fitting probabilistic models (e.g., normal distribution).
- **How:** MLE finds the parameter values that make the observed data most probable for the model.

`fitdistr(df$Score, "normal")`

- **Expected Range:**
 - Parameter estimates: Depends on the model (e.g., mean and variance for normal distribution).
 - Likelihood value: Can range widely but is used for model comparison rather than interpretation.
-

6. Regression Analysis

- **What:** A method to model the relationship between a dependent variable and one or more independent variables.
- **Why:** To predict the value of one variable based on others.
- **When:** Use when you want to predict a numeric outcome based on other variables (e.g., predicting score based on age and gender).
- **How:** The method estimates coefficients that explain the relationship between the dependent variable and predictors, along with significance tests.

`lm(Score ~ Age + Gender, data = df)`

- **Expected Range:**
 - Coefficients: Can be any real number (positive or negative).
 - R-squared (R^2): 0 to 1, where values closer to 1 indicate a better fit.
 - p-value for coefficients: 0 to 1. A value < 0.05 indicates a significant effect.

7. Classification Techniques (e.g., Logistic Regression)

- **What:** Techniques for predicting categorical outcomes.
- **Why:** To classify data into predefined categories (e.g., pass/fail).
- **When:** Use when the outcome is categorical, often binary (e.g., 1 = pass, 0 = fail).
- **How:** Logistic regression models the log-odds of the categorical outcome, providing coefficients, odds ratios, and p-values.

```
glm(Pass ~ Age + Gender, data = df, family = binomial)
```

- **Expected Range:**
 - **Odds ratio:** 0 to ∞ . A value > 1 indicates an increase in odds, and < 1 indicates a decrease.
 - **p-value for coefficients:** 0 to 1. A value < 0.05 indicates the predictor significantly impacts the outcome.
-

8. Clustering (e.g., K-Means)

- **What:** A technique for grouping data into clusters based on similarity.
- **Why:** To find natural groupings in data without predefined labels.
- **When:** Use when you want to segment data based on features (e.g., grouping students based on age and scores).
- **How:** K-means clustering partitions the data into k clusters, with centroids representing the mean of each cluster.

```
kmeans(df[, c("Age", "Score")], centers = 3)
```

- **Expected Range:**
 - **Cluster centroids:** Depends on the data's range.
 - **Within-cluster sum of squares (WSS):** ≥ 0 . A lower WSS indicates tighter clusters.
-

9. Association Rules Analysis (e.g., Apriori)

- **What:** A technique for discovering interesting relationships between variables in large datasets.
- **Why:** To identify patterns or associations in categorical data.
- **When:** Use when you want to find frequent itemsets or relationships (e.g., association between gender and department).
- **How:** The Apriori algorithm generates rules based on support, confidence, and lift metrics.

```
trans <- as(df[, c('Gender', 'Department')], "transactions")
```

```
rules <- apriori(trans, parameter = list(supp = 0.1, conf = 0.8))
```

- **Expected Range:**
 - **Support:** 0 to 1. Higher support indicates a more frequent rule.
 - **Confidence:** 0 to 1. Higher confidence suggests the rule is more reliable.
 - **Lift:** 0 to ∞ . A lift > 1 indicates a strong association.

Summary of Expected Ranges

Method	Expected Range
Chi-Square Test	Chi-square statistic: ≥ 0 ; p-value: 0 to 1
t-Test	t-statistic: $-\infty$ to $+\infty$ (typically -4 to 4); p-value: 0 to 1
ANOVA	F-statistic: ≥ 0 ; p-value: 0 to 1
Correlation Analysis	Correlation coefficient: -1 to 1; p-value: 0 to 1
Maximum Likelihood Estimation (MLE)	Varies based on the model and parameters
Regression Analysis	Coefficients: $-\infty$ to $+\infty$; R^2 : 0 to 1; p-value: 0 to 1
Classification (Logistic Regression)	Odds ratio: 0 to ∞ ; p-value: 0 to 1
Clustering	Cluster centroids depend on the data; WSS: ≥ 0
Association Rules	Support: 0 to 1; Confidence: 0 to 1; Lift: 0 to ∞

5. Implementing Analysis Techniques

Hypothesis Testing

Theory: Hypothesis testing is used to infer whether a certain condition holds true for a population based on sample data.

- **t.test()**: Performs a t-test to compare means between two groups.

```
t_test_result <- t.test(df$Age ~ df$Name)
```

Chi-Square Test

Theory: The Chi-Square test is used to examine whether there is a significant association between two categorical variables.

- **chisq.test()**: Performs a Chi-Square test to assess the association between categorical variables.

```
data_matrix <- matrix(c(10, 20, 20, 40), nrow = 2)
```

```
chi_sq_result <- chisq.test(data_matrix)
```

ANOVA (Analysis of Variance)

Theory: ANOVA is used to compare the means of multiple groups to see if at least one is significantly different.

- **aov()**: Performs ANOVA to compare means across multiple groups.

```
aov_result <- aov(Age ~ Name, data = df)
```

```
summary(aov_result)
```

Correlation Analysis

Theory: Correlation analysis measures the strength and direction of the linear relationship between two continuous variables.

- **cor()**: Computes the correlation coefficient between two continuous variables.

```
correlation <- cor(df$Age, df$Salary, use = "complete.obs")
```

Maximum Likelihood Estimation

Theory: Maximum likelihood estimation (MLE) is used to estimate parameters of a statistical model by maximizing the likelihood function.

- **optim()**: General-purpose optimization function used here for Maximum Likelihood Estimation. The function minimizes (or maximizes) a given function.

```
log_likelihood <- function(params) {
```

```
  mu <- params[1]
```

```
  sigma <- params[2]
```

```
  -sum(dnorm(df$Age, mean = mu, sd = sigma, log = TRUE))
```

```
}
```

```
result <- optim(c(mean(df$Age), sd(df$Age)), log_likelihood)
```

Regression Analysis

Theory: Regression analysis models the relationship between a dependent variable and one or more independent variables.

- **lm():** Fits a linear model to data, used for regression analysis.

```
lm_model <- lm(Salary ~ Age, data = df)
```

```
summary(lm_model)
```

Classification Techniques

Theory: Classification techniques categorize data into predefined classes based on input features.

- **rpart():** Constructs a decision tree for classification or regression. The method argument specifies the type of model.

```
library(rpart)
```

```
tree_model <- rpart(Salary ~ Age, data = df, method = "anova")
```

```
print(tree_model)
```

Clustering

Theory: Clustering groups data points into clusters based on their similarities.

- **kmeans():** Performs K-Means clustering, which partitions data into k clusters.

```
set.seed(123)
```

```
clusters <- kmeans(df[, c("Age", "Salary")], centers = 2)
```

Association Rules

Theory: Association rule mining is used to discover relationships between variables in large datasets, often used in market basket analysis.

- **arules:** A package for mining association rules. The apriori() function finds frequent itemsets and association rules.

```
library(arules)
```

```
data("Groceries") # Example dataset
```

```
rules <- apriori(Groceries, parameter = list(supp = 0.001, conf = 0.8))
```

```
inspect(rules)
```

Example:

Install and load necessary libraries

```
install.packages(c("MASS", "dplyr", "ggplot2", "cluster", "arules", "arulesViz"))
```

```
library(MASS)
```

```
library(dplyr)
```

```
library(ggplot2)
```

```
library(cluster)
```

```
library(arules)
```

```
library(arulesViz)
```

Create a sample dataset (50 rows)

```
set.seed(123)
```

```
df <- data.frame(
```

```
  Gender = sample(c("Male", "Female"), 50, replace = TRUE),
```

```
  Age = sample(18:40, 50, replace = TRUE),
```

```
  Score = round(rnorm(50, 70, 10), 2),
```

```
  Department = sample(c("CS", "Math", "Physics"), 50, replace = TRUE),
```

```
  Pass = sample(c(1, 0), 50, replace = TRUE) # 1 = pass, 0 = fail
```

```
)
```

```
Head(df)
```

```
Tail(df)
```

1. Chi-Square Test

Test if Gender and Department are independent

```
chisq_test_result <- chisq.test(table(df$Gender, df$Department))
```

```
print(chisq_test_result)
```

2. t-Test

Test if there is a difference in scores based on gender

```
t_test_result <- t.test(Score ~ Gender, data = df)
```

```
print(t_test_result)
```

3. ANOVA

Test if the mean score differs across departments

```
anova_result <- aov(Score ~ Department, data = df)
```

```
print(summary(anova_result))
```

4. Correlation Analysis

```
# Check correlation between Age and Score
```

```
correlation_result <- cor.test(df$Age, df$Score)
```

```
print(correlation_result)
```

5. Maximum Likelihood Estimation (MLE)

```
# Fit a normal distribution using MLE for the Score
```

```
mle_result <- fitdistr(df$Score, "normal")
```

```
print(mle_result)
```

6. Regression Analysis

```
# Perform linear regression of Score based on Age and Gender
```

```
regression_model <- lm(Score ~ Age + Gender, data = df)
```

```
print(summary(regression_model))
```

7. Logistic Regression (Classification)

```
# Predict Pass/Fail based on Age and Gender
```

```
logistic_model <- glm(Pass ~ Age + Gender, data = df, family = binomial)
```

```
print(summary(logistic_model))
```

8. K-Means Clustering

```
# Perform k-means clustering on Age and Score
```

```
set.seed(123)
```

```
kmeans_result <- kmeans(df[, c("Age", "Score")], centers = 3)
```

```
print(kmeans_result$centers)
```

```
# Plot clusters
```

```
df$cluster <- as.factor(kmeans_result$cluster)
```

```
ggplot(df, aes(x = Age, y = Score, color = cluster)) + geom_point() + ggtitle("K-Means Clustering")
```

9. Association Rules Analysis

Preparing the data for association rule mining

```
df_assoc <- df[, c("Gender", "Department")] # Keep only categorical columns for association rules
```

```
df_assoc[] <- lapply(df_assoc, as.factor) # Convert all columns to factors
```

Convert the data frame to a "transactions" object

```
trans <- as(df_assoc, "transactions")
```

Summary of transactions

```
summary(trans)
```

Adjusted thresholds for apriori algorithm

```
rules <- apriori(trans, parameter = list(supp = 0.05, conf = 0.6))
```

Check if rules are generated

```
if (length(rules) > 0) {
```

```
  # Inspect the generated rules
```

```
  inspect(rules)
```

```
  # Visualize the rules
```

```
  plot(rules, method = "graph", engine = "htmlwidget")
```

```
}else {
```

```
  cat("No rules generated! Try adjusting support and confidence values.\n")
```

```
}
```

1. Install and Load Libraries

```
install.packages(c("MASS", "dplyr", "ggplot2", "cluster", "arules", "arulesViz"))
```

```
library(MASS)
```

```
library(dplyr)
```

```
library(ggplot2)
```

```
library(cluster)
```

```
library(arules)
```

```
library(arulesViz)
```

- **install.packages(...):** This function installs the necessary libraries. These libraries help in performing statistical analyses, clustering, visualizations, and association rule mining.
 - **MASS:** Provides statistical functions like MLE (Maximum Likelihood Estimation).
 - **dplyr:** A data manipulation package.
 - **ggplot2:** A visualization package used for plotting graphs.
 - **cluster:** Used for clustering algorithms like K-Means.
 - **arules:** A package for association rule mining (Apriori algorithm).
 - **arulesViz:** A visualization package for association rules.
 - **library(...):** Loads the installed libraries for use in the current R session.
-

2. Creating the Sample Dataset

```
set.seed(123)
```

```
df <- data.frame(
```

```
  Gender = sample(c("Male", "Female"), 50, replace = TRUE),
```

```
  Age = sample(18:40, 50, replace = TRUE),
```

```
  Score = round(rnorm(50, 70, 10), 2),
```

```
  Department = sample(c("CS", "Math", "Physics"), 50, replace = TRUE),
```

```
  Pass = sample(c(1, 0), 50, replace = TRUE)
```

```
)
```

- **set.seed(123):** Ensures that random number generation is reproducible. 123 is the seed value, meaning you will always get the same random output when you run the code.
- **data.frame():** Creates a data frame `df`, a table-like structure in R where each column represents a variable, and each row represents an observation.
- **Variables:**
 - **Gender:** Randomly samples "Male" or "Female" for each row (50 rows in total). `replace = TRUE` means values can be repeated.
 - **Age:** Randomly samples ages between 18 and 40.

- **Score:** Generates 50 random numbers from a normal distribution (`rnorm()`) with a mean of 70 and a standard deviation of 10. `round(..., 2)` rounds the numbers to 2 decimal places.
 - **Department:** Randomly assigns a department ("CS", "Math", or "Physics").
 - **Pass:** Randomly assigns either 1 (pass) or 0 (fail) to each row.
-

3. Chi-Square Test

```
chisq_test_result <- chisq.test(table(df$Gender, df$Department))  
print(chisq_test_result)
```

- **chisq.test():** Performs a Chi-Square test to determine if there is a significant relationship between two categorical variables.
 - **table(df\$Gender, df\$Department):** Creates a contingency table (cross-tabulation) of the counts of Gender and Department.
 - **print():** Displays the result of the Chi-Square test.
-

4. t-Test

```
t_test_result <- t.test(Score ~ Gender, data = df)  
print(t_test_result)
```

- **t.test():** Performs a t-test to compare the means of Score between two groups (Gender). It tests if the means are significantly different.
 - **Score ~ Gender:** Formula notation in R indicating that Score is the dependent variable, and Gender is the independent variable.
 - **data = df:** Specifies the data frame being used.
 - **print():** Displays the t-test result.
-

5. ANOVA (Analysis of Variance)

```
anova_result <- aov(Score ~ Department, data = df)  
print(summary(anova_result))
```

- **aov():** Performs an ANOVA test to see if the mean Score differs significantly across different Departments.
 - **Score ~ Department:** Formula notation indicating that Score is the dependent variable, and Department is the independent variable.
 - **summary():** Summarizes the ANOVA result.
 - **print():** Displays the ANOVA summary.
-

6. Correlation Analysis

```
correlation_result <- cor.test(df$Age, df$Score)
```

```
print(correlation_result)
```

- **cor.test()**: Performs a correlation test to check the relationship between Age and Score.
 - **df\$Age, df\$Score**: These are the variables being compared for correlation.
 - **print()**: Displays the result of the correlation analysis.
-

7. Maximum Likelihood Estimation (MLE)

```
mle_result <- fitdistr(df$Score, "normal")
```

```
print(mle_result)
```

- **fitdistr()**: Fits a normal distribution to the Score data using Maximum Likelihood Estimation.
 - **"normal"**: Specifies the type of distribution to fit (in this case, normal distribution).
 - **print()**: Displays the result of the MLE, showing the estimated mean and standard deviation.
-

8. Linear Regression

```
regression_model <- lm(Score ~ Age + Gender, data = df)
```

```
print(summary(regression_model))
```

- **lm()**: Fits a linear regression model with Score as the dependent variable and Age and Gender as the independent variables.
 - **Score ~ Age + Gender**: Formula notation for the regression model.
 - **summary()**: Summarizes the regression result.
 - **print()**: Displays the summary of the linear regression model.
-

9. Logistic Regression (Classification)

```
logistic_model <- glm(Pass ~ Age + Gender, data = df, family = binomial)
```

```
print(summary(logistic_model))
```

- **glm()**: Fits a generalized linear model. In this case, logistic regression is used because the dependent variable (Pass) is binary.
 - **Pass ~ Age + Gender**: Formula notation for the logistic regression model.
 - **family = binomial**: Specifies that a logistic regression (binomial family) should be used.
 - **summary()**: Summarizes the logistic regression result.
 - **print()**: Displays the logistic regression summary.
-

10. K-Means Clustering

```
set.seed(123)
```

```
kmeans_result <- kmeans(df[, c("Age", "Score")], centers = 3)
```

```
print(kmeans_result$centers)
```

- **kmeans()**: Performs K-Means clustering on the columns Age and Score.
 - **centers = 3**: Specifies that we want 3 clusters.
 - **print(kmeans_result\$centers)**: Displays the centers of the 3 clusters.
-

11. Plotting K-Means Clustering

```
df$cluster <- as.factor(kmeans_result$cluster)
```

```
ggplot(df, aes(x = Age, y = Score, color = cluster)) + geom_point() + ggtitle("K-Means Clustering")
```

- **df\$cluster <- as.factor(kmeans_result\$cluster)**: Assigns the cluster result as a new column cluster in the data frame and converts it to a factor.
 - **ggplot()**: Creates a scatter plot of Age and Score with different colors for each cluster.
 - **aes(x = Age, y = Score, color = cluster)**: Aesthetic mapping for the plot.
 - **geom_point()**: Adds points to the scatter plot.
 - **ggtitle()**: Adds a title to the plot.
-

12. Association Rules (Apriori Algorithm)

```
df_assoc <- df[, c("Gender", "Department")]
```

```
df_assoc[] <- lapply(df_assoc, as.factor)
```

```
trans <- as(df_assoc, "transactions")
```

- **df[, c("Gender", "Department")]**: Subsets the data frame to include only the Gender and Department columns.
 - **lapply(..., as.factor)**: Converts each column in df_assoc to a factor.
 - **as(..., "transactions")**: Converts the data frame into a transaction format that the arules package can use for association rule mining.
-

13. Apriori Algorithm

```
rules <- apriori(trans, parameter = list(supp = 0.05, conf = 0.6))
```

- **apriori()**: Runs the Apriori algorithm to find association rules.
 - **trans**: The transaction object.
 - **parameter = list(supp = 0.05, conf = 0.6)**: Specifies the minimum support (0.05) and confidence (0.6) thresholds for rule generation.
-

14. Inspect and Plot Association Rules

```
if (length(rules) > 0) {
```

```
  inspect(rules)
```

```
  plot(rules, method = "graph", engine = "htmlwidget")
```

```
} else {
```

```
  cat("No rules generated! Try adjusting support and confidence values.\n")
```

- **length(rules) > 0**: Checks if any rules were generated.
- **inspect(rules)**: Displays the generated rules.
- **plot(rules, method = "graph", engine = "htmlwidget")**: Plots the rules using a graphical visualization.